

Institut Supérieur d'Informatique et de Multimédia de Sfax

Cours de

Algorithmique et Structures des Données

Auditoire : **Première année Licence en Sciences
Informatiques**

Plan du cours

Chapitre 1: Introduction à l'algorithmique

Chapitre 2: Les structures conditionnelles

Chapitre 3: Les structures itératives

Chapitre 4 : Les procédures et les fonctions

Chapitre 5 : Les tableaux et les tris

Chapitre 6 : Les chaînes de caractères

Chapitre 7 : Les pointeurs

Chapitre 8 : La récursivité

Chapitre 9 : Les enregistrements

Chapitre1 : Introduction à l'algorithmique

Concepts en informatique

- ❖ **Algorithme** : mot dérivé de «*al_Khwarizmi* », le nom du mathématicien du 9ème siècle et membre de la Maison de la sagesse de Bagdad.
- ❖ **Algorithmique** : le domaine qui étudie les algorithmes.
- ❖ Un **algorithme** est un texte constitué d'un ensemble de directives, appelées *instructions*, qui spécifient :
 - Les *opérations élémentaires* à exécuter
 - La façon dont elles s'*enchaînent*.
- ❖ Un algorithme est une suite finie d'instructions dont l'application permet de résoudre un certain problème et de produire un résultat.

Concepts en informatique...

- ❖ Un algorithme est un **pseudo-code** qui *ne dépend d'aucun langage* :
 - Ni du langage dans lequel il est implanté,
 - Ni de la machine qui exécute le programme.
- ❖ Un algorithme prend des *données en entrée*, exprime un *traitement* particulier et fournit des *données en sortie*.
- ❖ **Programme** : réalise ou implémente un algorithme en utilisant un langage de programmation compréhensible par l'ordinateur, peut être exécuté.

Structure générale d'un algorithme

Algorithme Nom_Algorithme

Constantes (*données du problème; constantes*)

C1 = Valeur1, C2 = Valeur2,..., Cn = Valeur n

Types

Type1 = définition

...

Variables (*données du problème; variables et types*)

V1 : Type1, V2 : Type2,... Vn : Type n

Début (*Traitement: instructions et résolution pas à pas*)

Instruction1

Instruction2

Les variables et les constantes

Problème :

Ecrire un algorithme permettant de calculer le périmètre et la surface d'un cercle de rayon 5.

1^{ère} solution :

Algorithme calcul

Début

Ecrire(" le périmètre = ", $2 * 3.14 * 5$)

Ecrire(" la surface = ", $3.14 * 5 * 5$)

Fin



Inconvénients:

Pour calculer la surface d'un autre cercle de rayon différent, il faut changer le code...

Analyse du problème:

- Lire les données en entrée (stocker la valeur du rayon dans une variable R),
- Faire les traitements nécessaires (utiliser la variable R dans le traitement),
- Fournir les données en sortie et afficher le résultat.

Les variables et les constantes...

2^{ème} solution :

Algorithme calcul

Constantes

$\pi = 3.14$

Variables

R, p, s : Réel

Début

Ecrire("Entrer le rayon du cercle : ")

Lire(R)

$P \leftarrow 2 * \pi * R$

$s \leftarrow \pi * R^2$

Ecrire ("Périmètre = ", p)

Ecrire ("Surface = ", s)

Fin.



Les variables et les constantes...

- ❖ En programmation, une **variable est un identificateur qui sert à repérer** un emplacement donné de la mémoire centrale. Cette notion nous permet de manipuler des valeurs sans nous préoccuper de l'emplacement qu'elles occupent effectivement en mémoire.
- ❖ Contrairement aux variables, les **constantes** sont des données dont la valeur reste **fixe** durant l'exécution du programme.

Exemple :

constante

Pi = 3.14

Types de variables

A chaque **variable** utilisée dans le programme, il faut **associer** un **type** qui permet de définir :

- l'ensemble des valeurs que peut prendre la variable
- l'ensemble des opérations qu'on peut appliquer sur la variable.

La syntaxe de la déclaration d'une variable est la suivante :

Variable 1, Variable 2, ... : Type

Les principaux types utilisés en algorithmique sont :

- le type entier
- le type réel
- le type caractère
- le type chaîne de caractères
- le type booléen.

Types de variables :Le type entier

Une variable est dite *entière* si:

1. Elle prend ses valeurs dans **Z** (ensemble des entiers relatifs)
2. Qu'elle peut supporter les opérations suivantes:

Opération	Notation
Addition	+
Soustraction	-
Multiplication	*
Division entière	Div
Modulo(reste de la division)	mod

Types de variables : Le type réel

Une variable est dite *réelle* si:

1. Elle prend ses valeurs dans **$\mathbf{R}$** (ensemble des nombres réels)
2. Qu'elle peut supporter les opérations suivantes:

Opération	Notation
Addition	+
Soustraction	-
Multiplication	*
Division réelle	/

Types de variables : Le type caractère

Dans une variable de type **caractère**, on stocke des :

- ✓ Lettres (minuscules et majuscules)
- ✓ Caractères spéciaux ('*', '£', '%', ...)
- ✓ Espaces: ' '
- ✓ Chiffres: '0'... '9'

Les opérateurs définis sur les données de type caractère sont :
=, ≠, <, <=, >, >=.

La comparaison entre les caractères se fait selon leur **code ASCII**:

' ' < '0' < '1' < 'A' < 'a' < '{'

Types de variables : Le type caractère

Le code ASCII

- ❖ Les caractères sont généralement codés sur 8 bits (1 octet)
- ❖ Chaque caractère possède son équivalent en code numérique : c'est le code ASCII (American Standard Code for Information Interchange)
- ❖ Le **code ASCII de base** représente les caractères sur 7 bits (c'est-à-dire 128 caractères possibles, de 0 à 127). Il a été mis au point pour la langue anglaise, il ne contient donc pas de caractères accentués, ni de caractères spécifiques à une langue. Pour coder ce type de caractère il faut recourir à un autre code : c'est le **code ASCII étendu** à 8 bits (un octet)
 - ❖ Les codes de 48 à 57 représentent les chiffres '0', '1', ..., '9'
 - ❖ Les codes de 65 à 90 représentent les majuscules 'A', 'B', ... 'Z'
 - ❖ Les codes de 97 à 122 représentent les minuscules 'a', 'b', ...

Types de variables : Le type caractère

Le code ASCII...

000 (nul)	016 ► (dle)	032 sp	048 0	064 @	080 P	096 `	112 p
001 ☉ (soh)	017 ◀ (dc1)	033 !	049 1	065 A	081 Q	097 a	113 q
002 ● (stx)	018 ↑ (dc2)	034 "	050 2	066 B	082 R	098 b	114 r
003 ▼ (etx)	019 !! (dc3)	035 #	051 3	067 C	083 S	099 c	115 s
004 ♦ (eot)	020 ¶ (dc4)	036 \$	052 4	068 D	084 T	100 d	116 t
005 ♣ (enq)	021 § (nak)	037 %	053 5	069 E	085 U	101 e	117 u
006 ♠ (ack)	022 − (syn)	038 &	054 6	070 F	086 V	102 f	118 v
007 • (bel)	023 ↓ (etb)	039 '	055 7	071 G	087 W	103 g	119 w
008 ■ (bs)	024 ↑ (can)	040 (	056 8	072 H	088 X	104 h	120 x
009 (tab)	025 ↓ (em)	041)	057 9	073 I	089 Y	105 i	121 y
010 (lf)	026 (eof)	042 *	058 :	074 J	090 Z	106 j	122 z
011 ♂ (vt)	027 − (esc)	043 +	059 ;	075 K	091 [	107 k	123 {
012 ♀ (np)	028 L (fs)	044 ,	060 <	076 L	092 \	108 l	124
013 (cr)	029 ↔ (gs)	045 −	061 =	077 M	093]	109 m	125 }
014 ♀ (so)	030 ▲ (rs)	046 .	062 >	078 N	094 ^	110 n	126 ~
015 ♂ (si)	031 ▼ (us)	047 /	063 ?	079 O	095 _	111 o	127 △

Table code ASCII 0-127

Types de variables : Le type booléen

- Une variable logique ne peut prendre que les **valeurs** **VRAI** ou **FAUX**
- Elle intervient dans l'évaluation d'une condition
- Les principales **opérations** définies sur les variables de type logique sont :
 - la négation (**NON**),
 - l'intersection (**ET**)
 - l'union (**OU**).

Déclarer une constante

constante <identificateurs> = <expression>

Fonction

Instruction permettant de **réserver de l'espace mémoire** pour stocker des données dont la **valeur est fixée pour tout l'algorithme**

Exemple

constante MAX = 100
 DOUBLEMAX = MAX × 2

Déclarer une Variable

Variable

<identificateurs>: **type**

Fonction

Instruction permettant de **réserver de l'espace mémoire** pour **stocker provisoirement des données** (dépend du type de ces données: entier, réels, caractères, chaîne de caractères ou booléen)

Exemples

Variable val: entier

nom: chaîne de caractères

Remarques :

- ❖ Le nom d'une variable doit être :
 - Unique pour chaque variable
 - Commence toujours par une lettre, les chiffres sont autorisés à condition de ne pas les mettre au début du nom
- ❖ Toute variable doit être déclarée avant son utilisation
- ❖ La lisibilité d'un algorithme dépend étroitement du choix des noms des variables qui doivent être

simples et significatifs

Saisir une donnée

Lire (<liste des variables>)

Fonction

Instruction permettant de **placer en mémoire les informations fournies par l'utilisateur**

Exemples

Lire (val)

Lire (nom, prenom)

Afficher une donnée

Ecrire (<liste des variables, de constantes ou d'expressions>)

Fonction

Instruction permettant de **visualiser les informations placées en mémoire.**

Exemples

Ecrire (val, "est différent de 0")

Types d'opérateurs

❖ Il existe plusieurs types d'opérateurs :

- Les opérateurs arithmétiques
- Les opérateurs de comparaison
- Les opérateurs logiques
- L'opérateur de concaténation de chaînes
- L'opérateur d'affectation (□)

Types d'opérateurs...

❖ Les opérateurs arithmétiques

permettent d'effectuer des opérations arithmétiques entre des opérandes numériques (entiers, réels)

- Opérations élémentaires : **+**, **-**, *****, **div**, **mod**, **/**
- changement de signe : **" - "**
- élévation à la puissance : **" ^ "**

❖ Les opérateurs de comparaison **=**, **s**, **>**, **>=**, **<**, **<=**

permettent de comparer deux opérandes, le résultat est une valeur booléenne. Ils s'appuient sur des **relations d'ordre** : Ordre naturel (entiers ou réels) et Ordre lexicographique ASCII (caractères, chaînes de caractères)

Types d'opérateurs...

❖ Les opérateurs logiques : **NON, ET, OU**

- combinent des opérandes booléennes pour former des expressions logiques plus complexes

❖ L'opérateur de concaténation de chaînes de caractères (+)

- permet d'ajouter une chaîne à la fin d'une autre chaîne, le résultat est une chaîne

❖ L'opérateur d'affectation ()

- c'est l'opérateur qui attribue une valeur à une variable

Exemple : A 6

signifie mettre la valeur 6 dans la case mémoire identifiée par A

Remarque : La valeur de l'expression à droite du signe d'affectation doit être du même type ou de type compatible avec celui de la variable à gauche.

Affecter une valeur à une variable...

Comparons maintenant les deux algorithmes suivants :

Exemple n°1

Début

X ←

'R'

Y ←

'X'

Fin

Exemple n°2

Début

X ← 'R'

Y ← X

Fin

Notion d'expression

- ❖ Une expression est une combinaison de variables, constantes et opérateurs.
- ❖ Une expression exprime un calcul (expression arithmétique) ou une relation (expression logique)
- ❖ Exemples : $a+b$; $5*x-(y/z^2)$; **NON** a **OU** b **ET** c
- ❖ Durant le déroulement de l'algorithme, toutes les expressions sont évaluées
- ❖ Lors de l'évaluation d'une expression, **la priorité de chaque opérateur** permet de définir **l'ordre** selon lequel les différentes opérations seront exécutées
- ❖ Pour lever l'ambiguïté ou modifier cet ordre, on utilise des **parenthèses**

Notion d'expression...

Priorité des opérateurs dans l'évaluation d'une expression

Priorité	Type	Opérateur
1	Signe négatif (moins unaire)	-
2	Parenthèses	()
3	Puissance	^
4	Négation	NON
5	Multiplication / Division	* / div mod
6	Addition / Soustraction	+ -
7	Op. relationnels	> < >= <=
8	Égalité / Inégalité	= s
9	ET relationnel	ET
10	OU relationnel	OU
11	L'opérateur d'affectation	=

Remarque : Si une expression contient plusieurs opérateurs ayant la même priorité, on commence par celui situé le plus à **gauche**

Expression arithmétique

L'ordre selon lequel se déroule chaque opération de calcul est important.

On se sert des parenthèses et des relations de priorité entre les opérateurs pour éviter l'ambiguïté dans l'écriture.

Exemple: $x * 5.3 / (2 + \pi)$

L'ordre de priorité des opérateurs arithmétiques est le suivant:

Priorité	Opérateurs
1	- Signe négatif (unaire)
2	$()$
3	$\wedge$
4	$*$, $/$, div, mod
5	$+$, $-$

Expressions logiques

Les opérations définies sur les variables logiques sont : la négation (NON), l'intersection (ET) et l'union (OU)

L'application de ces opérateurs se fait selon la table de vérité suivante:

A	B	NON (A)	A ET B	A OU B
Vrai	Vrai	Faux	Vrai	Vrai
Vrai	faux	Faux	Faux	Vrai
Faux	Vrai	Vrai	Faux	Vrai
Faux	faux	vrai	faux	faux

Expressions et opérateurs

Les expressions logiques sont des combinaisons entre des variables et/ou des constantes à l'aide d'opérateurs relationnels ($=$, $<$, ...) et/ou des combinaisons entre les variables et/ou des constantes logiques à l'aide d'opérateurs logiques (NON, ET, OU).

L'ordre de priorité des ces opérateurs est le suivant :

Priorité	Opérateurs
1	NON
2	ET
3	OU

Chapitre2 : Les structures conditionnelles

Types des structures de contrôle

Il n'y a que **deux formes possibles** pour un test :

- ❖ Les instructions conditionnelles simples/ alternatives ou emboîtées
- ❖ Les instructions conditionnelles à choix multiples

L'instruction conditionnelle simple/ alternative

Si *expression logique* **Alors**

Instructions 1

[Sinon

Instructions 2]

Finsi

Si l'expression logique (condition) prend la valeur **vrai**, le premier bloc d'instructions est exécuté; si elle prend la valeur **faux**, le second bloc est exécuté (s'il est présent (instruction conditionnelle alternative) , sinon rien).

L'instruction conditionnelle simple : *Exemple 1*

Algorithme SimpleOuDouble

{cet algorithme saisit une valeur entière et affiche son double si cette donnée est inférieure à un seuil donné}

Constante seuil = 10

Variable val: entier

Début

Ecrire(" donnez un nombre ")

lire(val)

si val < seuil *alors*

 Ecrire (" voici son double " , val *2)

sinon Ecrire (" la valeur est

inchangée " , val) *finsi*

Fin

Début

Ecrire(" donnez un nombre ")

lire(val)

si val < seuil *alors*

 val ← val*2

finsi

Ecrire(" la valeur est " , val)

Fin

Une autre
écriture de
l'algorithme

L'instruction conditionnelle simple : *Exemple 2*

Ecrire un algorithme qui calcule le salaire d'un employé à partir du nombre d'heures travaillées, du taux horaire et du nombre d'années de service. Les employés ayant une ancienneté de plus de 10 ans bénéficient d'une allocation supplémentaire de 35 D

Algorithme Calc_Salaire

Variables nh, th, anc, salaire : Réel

Début

Ecrire("Nombre d'heures travaillées : ") Lire(nh)

Ecrire("Taux horaire : ") Lire(th)

Ecrire("Ancienneté : ") Lire(anc)

Salaire $nh * th$

Si (anc > 10) **Alors**

salaire salaire + 35

FinSi

Ecrire ("Salaire de l'employé = ", salaire)

Fin.

L'instruction conditionnelle emboîtée

Quand la condition se complique :

Si expression1 **Alors**

Instructions 1

Sinon

Si expression2 **Alors**

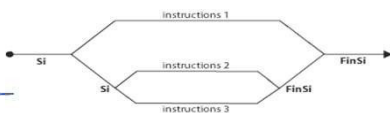
Instructions 2

Sinon

Instructions 3

Finsi

Finsi



Structures de tests imbriqués

📖 **Notons** que les suites d'instructions peuvent comporter à leur tour d'autres instructions conditionnelles (si ... alors ...).

📖 **Notons** que dans un schéma conditionnel :

- A chaque « si » doit correspondre un « fin si »
- Un « fin si » revient au dernier « si » qui n'a pas de « fin si »

L'instruction conditionnelle emboîtée : *Exemple 1*

Problème: on veut afficher le résultat selon une note

- **reçu avec mention** si la note est supérieure ou égale à 12
- **passable** si elle est supérieure ou égale à 10 et inférieure à 12
- **insuffisant** dans les autres cas.

Début

si note ≥ 12 *alors*

Ecrire(" reçu avec mention ")

sinon si note ≥ 10 *alors*

Ecrire(" passable ")

sinon

Ecrire(" insuffisant ")

finsi

finsi

Fin

L'instruction conditionnelle emboîtée : *Exemple 2*

Ecrire un algorithme qui calcule et affiche la valeur absolue d'un entier quelconque lu au clavier.

Algorithme Val_Abs

Variables

x, va : Entier

Début

Ecrire("Entrer un
entier : ")

Lire(x)

Si ($x \geq 0$) **Alors**

$va \leftarrow x$

Sinon

$va \leftarrow -x$

FinSi

Ecrire ("|", x, "| =
", va)

L'instruction conditionnelle emboîtée : *Exemple 3*

Ecrire un algorithme qui lit un entier et affiche ensuite s'il est pair ou impair.

Algorithme pair_impair

Variables

x : Entier

Début

Ecrire("Entrer un
entier : ") Lire(x)

Si (x Mod 2 = 0)

Alors

Ecrire("c'est un
entier pair")

Sinon

Ecrire("c'est un
entier impair")

FinSi

Sélection sur choix multiples

Si l'on doit comparer une même variable avec plusieurs valeurs, comme par exemple:

Début

```
Si (abrev =  
    "M" ou  
    abrev =  
    "Mr" ) Alors  
    s  
    Ecrire("  
    Monsieur  
    ")
```

Sinon

```
Si abrev = "Mme" Alors
```

```
    Ecrire(" Madame ")
```

Sinon

```
    Ecrire(" Monsieur, Madame" )
```

Finsi

Finsi

Fin

Sélection sur choix multiples ...

Syntaxe:

Selon sélecteur **Faire**

Liste de valeurs 1 : < Séquence d'instructions 1>

Liste de valeurs 2 : < Séquence d'instructions 2>

...

Liste de valeurs n : < Séquence d'instructions n>

[**Sinon**

<Autre

Séquence

d'instruction

ns>]

FinSelon

- Le sélecteur est une variable ou une expression de type **scalaire**

Sélection sur choix multiples : *Exemple 1*

Selon abbrev **Faire**

"M" , "Mr" : *Ecrire*(" Monsieur ")

"Mme": *Ecrire*(" Madame
")

Sinon

Ecrire(" Monsieur, Madame")

FinSelon

Sélection sur choix multiples : *Exemple 2*

Ecrire un algorithme qui permet de lire un numéro compris entre 1 et 12 et d'afficher le nom du mois correspondant. Si le numéro entré est en dehors de cet intervalle, un message d'erreur doit être affiché.

Algorithme mois

Variables

n : Entier

Début

Ecrire("Entre
r le numéro
du mois : ")

Lire(n)

Selon n

Faire

1 :

Ecrire("j
anvier")

2 :

Ecrire("f
évrier")

Sélection sur choix multiples : *Exemple 2...*

L'exercice précédent peut être résolu en utilisant plusieurs instructions imbriquées, mais l'algorithme sera très lourd :

« Si »

Algorithme mois {version Si }

Variables

n : Entier

Début

Ecrire("Entrer le numéro du mois : ") Lire(n)

Si (n = 1) Alors

Ecrire("janvier")

Sinon Si (n = 2) Alors

Ecrire("février")

Sinon Si (n = 3) Alors

Ecrire("mars")

Sinon Si (n = 4) Alors

...

Fin

Chapitre 3 : Les structures itératives

Introduction

Plutôt de recopier n fois les mêmes instructions, on peut demander à l'ordinateur d'exécuter n fois le même traitement.

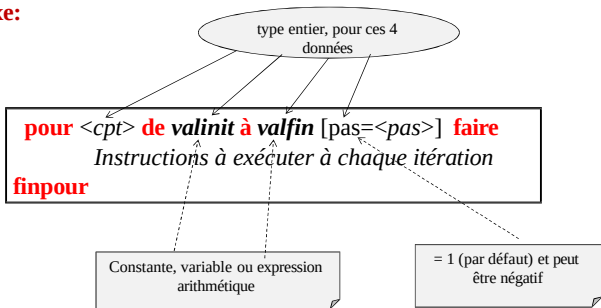
Il existe deux catégories d'itérations:

- ❖ itérations déterministes: le nombre d'itérations est défini à l'entrée de la boucle.
- ❖ itérations indéterministes: l'exécution de la prochaine itération de la boucle est conditionnée par une expression booléenne.

Structures itératives : La boucle **pour**

C'est l'instruction **pour** qui permet de faire des **boucles déterministes**. Il s'agit de répéter une suite d'instructions un certain nombre de fois.

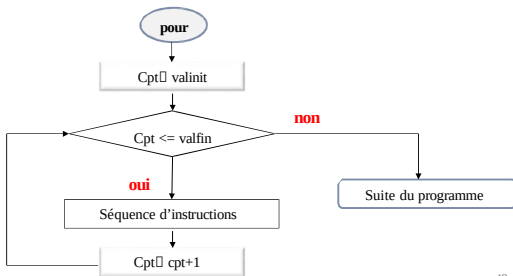
Syntaxe:



Structures itératives : La boucle **pour**...

Principe de fonctionnement :

Le compteur (variable de contrôle) prend la valeur initiale au moment d'accès à la boucle puis, à chaque parcours, il passe **automatiquement** à la valeur suivante dans son domaine jusqu'à atteindre la valeur finale.



Structures itératives : La boucle **pour** : *Exemple 1*

Pour i de 1 à 5 Faire

Ecrire($i \times 10$)

FinPour

Cette
boucle
affiche
respectivement les
nombres
10, 20, 30,
40 et 50.

Remarques :

- Une

Structures itératives : La boucle **pour** : *Exemple 2*

Dans une boucle « pour », l'évolution du compteur peut se faire dans le sens **décroissant** comme dans l'exemple suivant :

Pour i de 5 à 1 [pas = -1] **Faire**

Ecrire(i*10)

FinPour

Dans ce cas, le compteur i sera **décrémenté** après chaque parcours.

Structures itératives : La boucle **pour** : *Exemple 3*

Calcul de la somme des n nombres saisis.

Algorithme Somme

Variable *n, cpt: entier*
valeur, total: réel

Début

Ecrire(" combien de valeurs voulez-vous saisir?")

Lire(*n*)

total $\leftarrow 0$

pour *cpt*

de 1 à n

faire

E

c

r

i

r

e

Structures itératives : La boucle **pour** : *Exemple 4*

Ecrire un algorithme qui lit un entier positif n puis affiche tous ses diviseurs.

Algorithme Diviseurs { *version pour* }

Variables

n, i : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

Pour i de 1 à n **Faire**

 Si ($n \bmod i = 0$) Alors

 Ecrire(i)

 FinSi

FinPour

Fin.

Structures itératives : La boucle **pour** : *Exemple 5*

Exemple: Ecrire un algorithme qui lit un entier positif n puis calcule et affiche son factoriel selon la formule $n! = 1 * 2 * \dots * n$.

Algorithme Facto { *version pour* }

Variables

n, i, f : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

$f \leftarrow 1$

Pour i de 2 à n **Faire**

$f \leftarrow f * i$

FinPour

Ecrire(n , " ! = ", f)

Fin.

Structures itératives : La boucle **répéter...jusqu'à...**

Syntaxe:

répéter

(ré)affectation des variables de condition

Instructions à exécuter à chaque boucle

jusqu'à *<condition>*

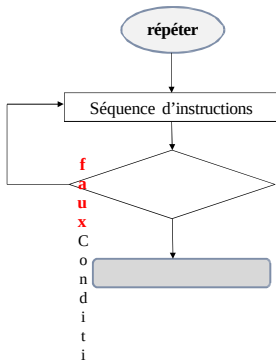
Principe de fonctionnement

La séquence d'instructions est exécutée une première fois, puis l'exécution se répète jusqu'à ce que la condition de sortie soit vérifiée.

Remarque: la boucle répéter est typique pour les saisies avec vérification.

Structures itératives : La boucle **répéter...jusqu'à...**

Une boucle « répéter » s'exécute toujours au moins une fois.



Structures itératives : La boucle **répéter...jusqu'à...**

Exemple 1

Algorithme Somme

Constante $stop = -1$

Variables $valeur, total$: réel

Début

t

o

t $Ecrire(" donnez une valeur différente de" , stop)$

a $Lire(valeur)$

l $total \leftarrow total + valeur$

Jusqu'à $valeur = stop$

$Ecrire(" la somme est " , total + 1)$

Fin r

$é$

p

en

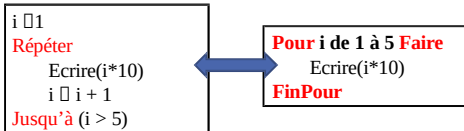
e

r

// mauvais choix de boucle, il est préférable d'utiliser tant que

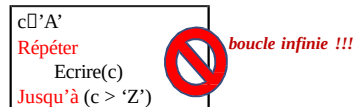
Structures itératives : La boucle **répéter...jusqu'à...**

Exemple 2



Remarques:

- Contrairement à une boucle « pour », dans une boucle « répéter », l'initialisation et l'avancement du compteur doivent être gérés manuellement par le programmeur.
- Dans une boucle « répéter jusqu'à », il faut toujours s'assurer que la condition de sortie sera vérifiée après un nombre fini de parcours. Sinon, c'est une *boucle infinie* comme dans le cas suivant :



Structures itératives : La boucle **répéter...jusqu'à...**

Exemple 3

Réécrire l'algorithme diviseurs en remplaçant la boucle « pour » par une boucle « répéter jusqu'à »

Algorithme Diviseurs (* version répéter *)

Variables n, i : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

i ← 1

Répéter

Si (n Mod i = 0) **Alors**

Ecrire(i)

FinSi

i ← i + 1

Jusqu'à (i > n)

Fin.

Structures itératives : La boucle **répéter...jusqu'à...**

Exemple 4

Réécrire l'algorithme facto en remplaçant la boucle « pour » par une boucle « répéter jusqu'à »

Algorithme Facto (* version répéter *)

Variables

n, f, i : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

$f \leftarrow 1$

$i \leftarrow 1$

Répéter

$f \leftarrow f * i$

$i \leftarrow i + 1$

Jusqu'à ($i >$

n)

Ecrire(n, "!" =

",f)

Fin

Structures itératives : La boucle **tant que**

Quand le nombre d'itérations n'est pas connu, alors on peut utiliser la boucle tant que.

Syntaxe:

initialisation des variables de condition

Tant que condition **faire**

*Instructions à exécuter à chaque boucle
réaffectation des variables de condition*

Fintant que

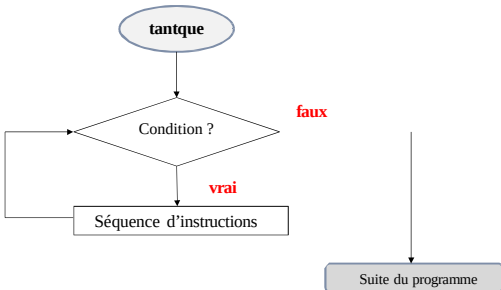
Principe de fonctionnement

Le traitement est exécuté aussi longtemps que la condition est vérifiée. Si dès le début cette condition est fausse, le traitement ne sera exécuté aucune fois.

Remarque: si la condition est fausse dès le départ, le traitement n'est **jamais** exécuté.

Structures itératives : La boucle **tant que...**

Une boucle « tantque » peut s'exécuter 0, 1 ou n fois.



Structures itératives : La boucle **tant que**

Exemple 1

```
i ← 1  
TantQue (i ≤ 5) Faire  
    Ecrire(i*10)  
    i ← i + 1  
FinTQ
```



```
Pour i de 1 à 5 Faire  
    Ecrire(i*10)  
FinPour
```

Structures itératives : La boucle **tant que**

Exemple 2

Réécrire l'algorithme diviseurs en remplaçant la boucle « répéter » par une boucle « tantque »

Algorithme Diviseurs (* version tantque *)

Variables

n, i : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

i ← 1

TantQue (i ≤ n) **Faire**

Si (n Mod i = 0) **Alors**

 Ecrire(i)

FinSi

i ← i + 1

FinTQ

Fin.

Structures itératives : La boucle **tant que**

Exemple 3

Réécrire l'algorithme facto en remplaçant la boucle « répéter » par une boucle « tantque ».

Algorithme Facto (* version tantque *)

Variables

n, f, i : Entier

Début

Ecrire("Entrer un entier positif : ") Lire(n)

$f \leftarrow 1$

$i \leftarrow 2$

TantQue ($i \leq n$) **Faire**

$f \leftarrow f * i$

$i \leftarrow i + 1$

FinTQ

Ecrire(n, "!" =

",f)

Fin.

Tant que vs. Pour

Implicitement, l'instruction **pour** :

- Initialise la variable de la boucle (compteur)
- incrémente cette variable à chaque pas
- vérifie que cette variable ne dépasse pas la borne supérieure

Explicitement, l'instruction **tant que** doit:

- Initialiser un compteur
- incrémenter le compteur à chaque pas
- vérifier que cette variable ne dépasse pas la borne supérieure

Le problème d'une boucle: il faut en sortir

tant que A faire B

répéter B tant que A

Les instructions de B doivent modifier au moins une variable de l'expression logique A.

Mauvais exemple:



Début

val1 $\leftarrow$ 2

val2 $\leftarrow$ 3

Tant que val1 < 100 **faire**

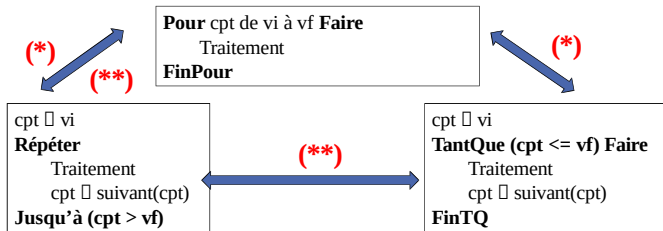
val2 $\leftarrow$ val2 * val1

fintantque

Fin

C'est l'expression logique A
(et elle seule!) qui en prenant
la valeur faux provoque
l'arrêt de la boucle.

Passage d'une structure itérative à une autre



(*) : Le passage d'une boucle « répéter » ou « tantque » à une boucle « pour » n'est possible que si le nombre de parcours est connu à l'avance.

()** : Lors du passage d'une boucle « pour » ou « tantque » à une boucle « répéter », faire attention aux cas particuliers (le traitement sera toujours exécuté au moins une fois).

Choix de la structure itérative

